

TOKYO, JAPAN / DECEMBER 11-13, 2025

EROFS and containers



東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

EROFS recap



What is EROFS?

Enhanced Read-Only File System (originally started in late 2017), since Linux 5.4 (2019).

A block-based immutable filesystem with a simple, unencoded core disk format:

- Similar to an archive format (e.g., tar, zip, cpio) but with more advanced features;
- However, it uses fixed filesystem block sizes: data is strictly aligned on disk within filesystem blocks for extreme performance, and that enables other possibilities:
  - FSDAX (passthrough host memory to guest to avoid guest page cache)
  - Direct I/O (load data directly via DMA from underlay storage)
  - Chunk-basd deduplication; also can leverage deduplication across the storage stack:
    e.g. reflink in the underlay filesystem (XFS, BTRFS, etc.)
- File-backed mounts (6.12+)
- It can resist malicious untrusted data by design, and currently fuzzed pretty well against malicious crafted images

Applications:

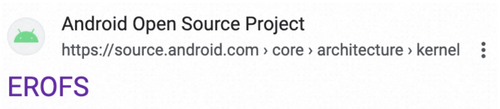
composefs containerd gVisor nerdbox

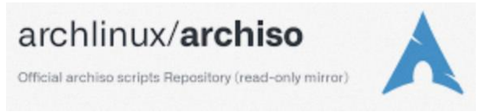
What is EROFS?

A full-featured EROFS supports with the following optional features:

- LZ4 / LZMA/ DEFLATE / Zstd transparent compression on a per-file basis;
- Tail-end data packing for compressed data (aka. fragments);
- A variant-CDC (un)compresed data deduplication;
- Metadata compression

Applications:













current EROFS container use cases

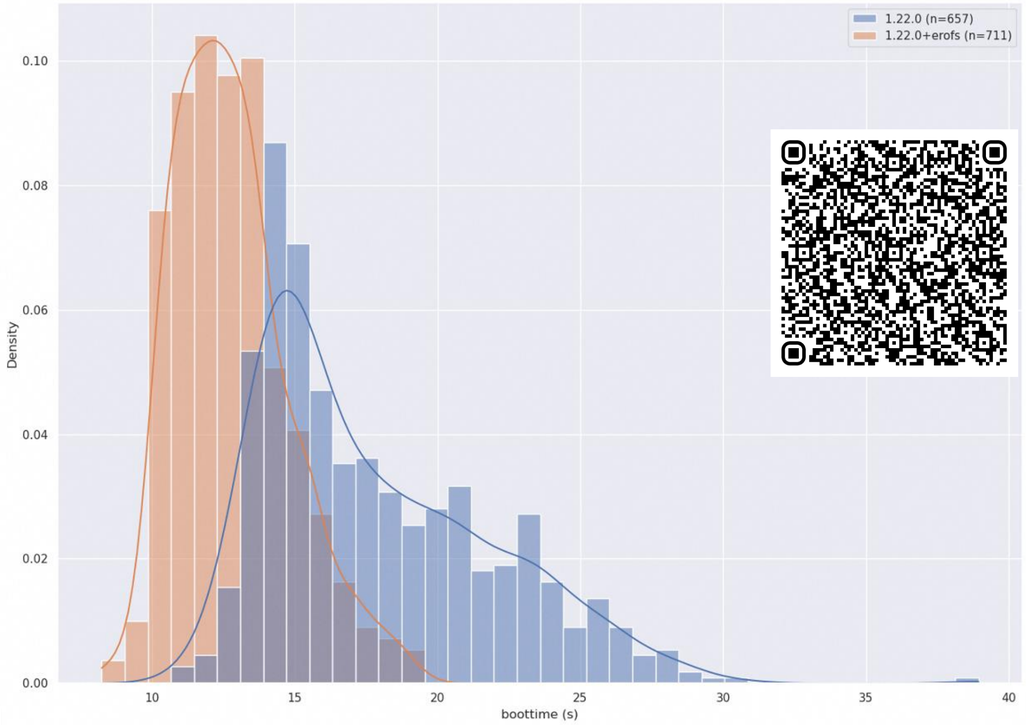


TOKYO, JAPAN / DEC. 11-13, 2025



Immutable OSes

“A recent addition of EROFS made it start even faster while using a 46% smaller root EBS volume.”



AWS Bottlerocket OS (EXT4 => EROFS)

Discoverable Disk Images
“Each partition contains a standard Linux filesystem (e.g.: erofs), so again this will not be redefined here.”

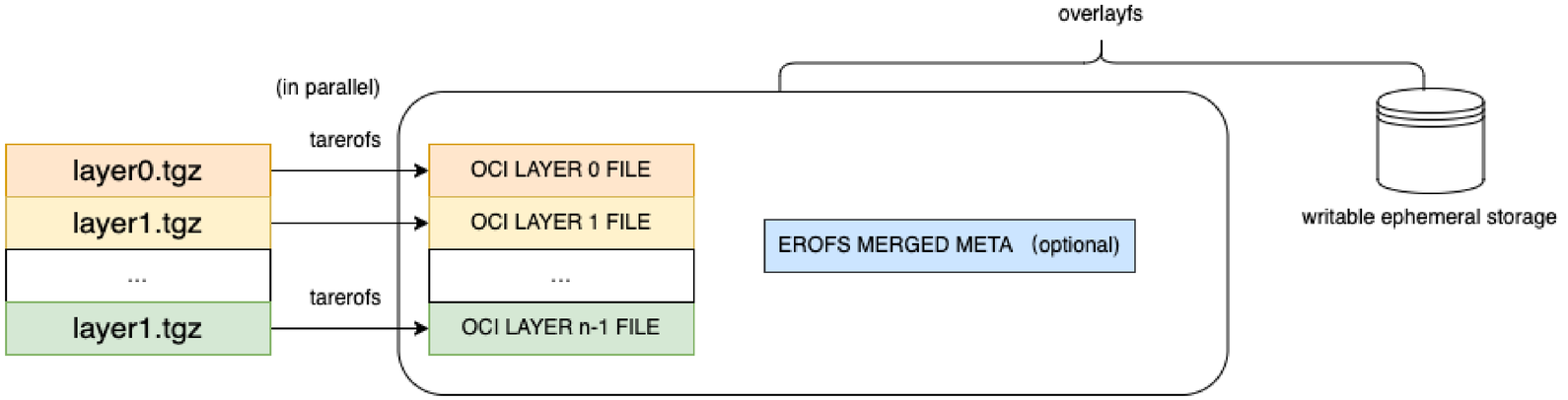
Amazon Linux 2023 Attestable AMIs

Read-only root filesystem — Use Enhanced Read-Only File System (erofs) with dm-verity protection to ensure that the root filesystem cannot be modified and maintains cryptographic integrity verification.

Containerd built-in EROFS Integration

Containerd now has native EROFS support:

- Introduced as a highlight feature in the containerd 2.1 release;
- Currently used primarily to improve the OCI image experience;
  - better performance by avoiding file unpacking to the underlying filesystem;
  - Ensures runtime data integrity through fsverity.
- File-backed mounts without block devices.

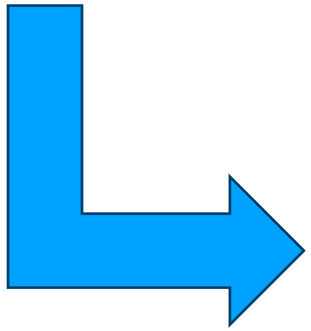


Containerd EROFS built-in adaption

File-backed mounts:

```
/dev/loop0 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/8 ext4 rw,relatime 0 0
/dev/loop1 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/7 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/dev/loop2 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/6 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/dev/loop3 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/5 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/dev/loop4 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/4 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/dev/loop5 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/3 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/dev/loop6 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/2 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/dev/loop7 /run/containerd/io.containerd.mount-manager.v1.bolt/t/376/1 erofs ro,relatime,user_xattr,acl,cache_strategy=readaround 0 0
overlay /run/containerd/io.containerd.runtime.v2.task/default/erofs/rootfs overlay rw,relatime,lowerdir=/run/containerd/io.containerd.m
ount-manager.v1.bolt/t/376/7:/run/containerd/io.containerd.mount-manager.v1.bolt/t/376/6:/run/containerd/io.containerd.mount-manager.v1
.bolt/t/376/5:/run/containerd/io.containerd.mount-manager.v1.bolt/t/376/4:/run/containerd/io.containerd.mount-manager.v1.bolt/t/376/3:/
run/containerd/io.containerd.mount-manager.v1.bolt/t/376/2:/run/containerd/io.containerd.mount-manager.v1.bolt/t/376/1,upperdir=/run/co
ntainerd/io.containerd.mount-manager.v1.bolt/t/376/8/upper,workdir=/run/containerd/io.containerd.mount-manager.v1.bolt/t/376/8/work 0 0
```

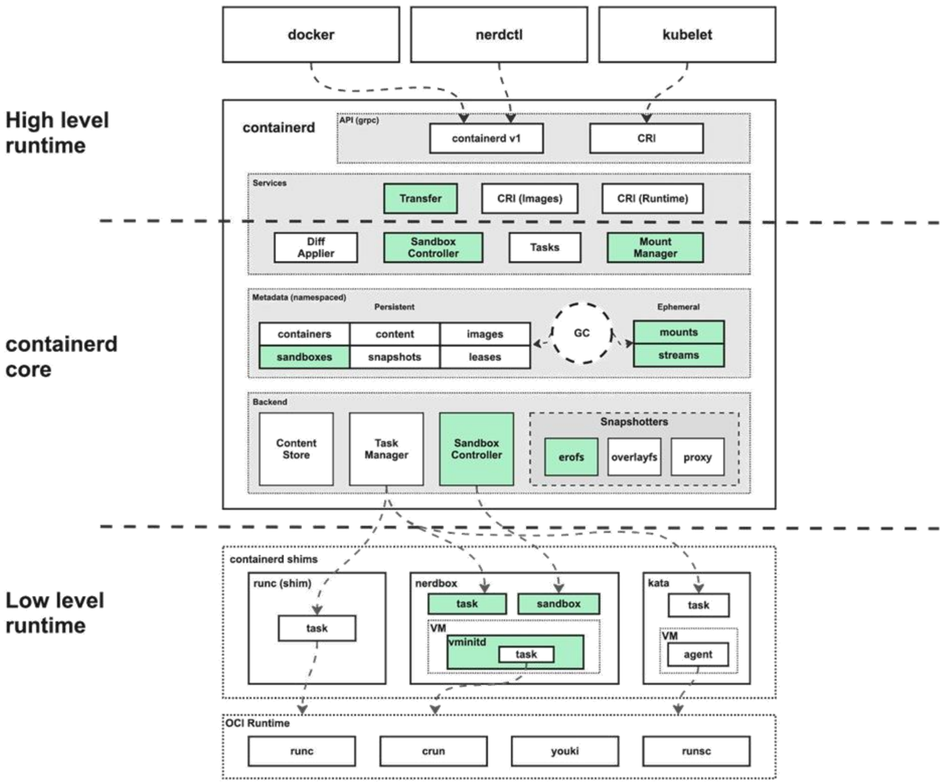
device on Linux kernels w/o file-backed mounts



```
/dev/loop0 /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/8 ext4 rw,relatime 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/45/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/7 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/44/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/6 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/43/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/5 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/42/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/4 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/41/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/3 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/40/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/2 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/39/layer.erofs /run/containerd/io.containerd.mount-manager.v1.bolt/t/8/1 erofs r
o,relatime,user_xattr,acl,cache_strategy=readaround 0 0
overlay /run/containerd/io.containerd.runtime.v2.task/default/erofs/rootfs overlay rw,relatime,lowerdir=/run/containerd/io.containerd.mount-manag
er.v1.bolt/t/8/7:/run/containerd/io.containerd.mount-manager.v1.bolt/t/8/6:/run/containerd/io.containerd.mount-manager.v1.bolt/t/8/5:/run/contain
erd/io.containerd.mount-manager.v1.bolt/t/8/4:/run/containerd/io.containerd.mount-manager.v1.bolt/t/8/3:/run/containerd/io.containerd.mount-manag
er.v1.bolt/t/8/2:/run/containerd/io.containerd.mount-manager.v1.bolt/t/8/1,upperdir=/run/containerd/io.containerd.mount-manager.v1.bolt/t/8/8/upp
er,workdir=/run/containerd/io.containerd.mount-manager.v1.bolt/t/8/8/work,uuid=on 0 0
```

device on kernels with file-backed mounts (6.12+)

Containerd EROFS snapshotter



EROFS snapshoter improvements in v2.2:

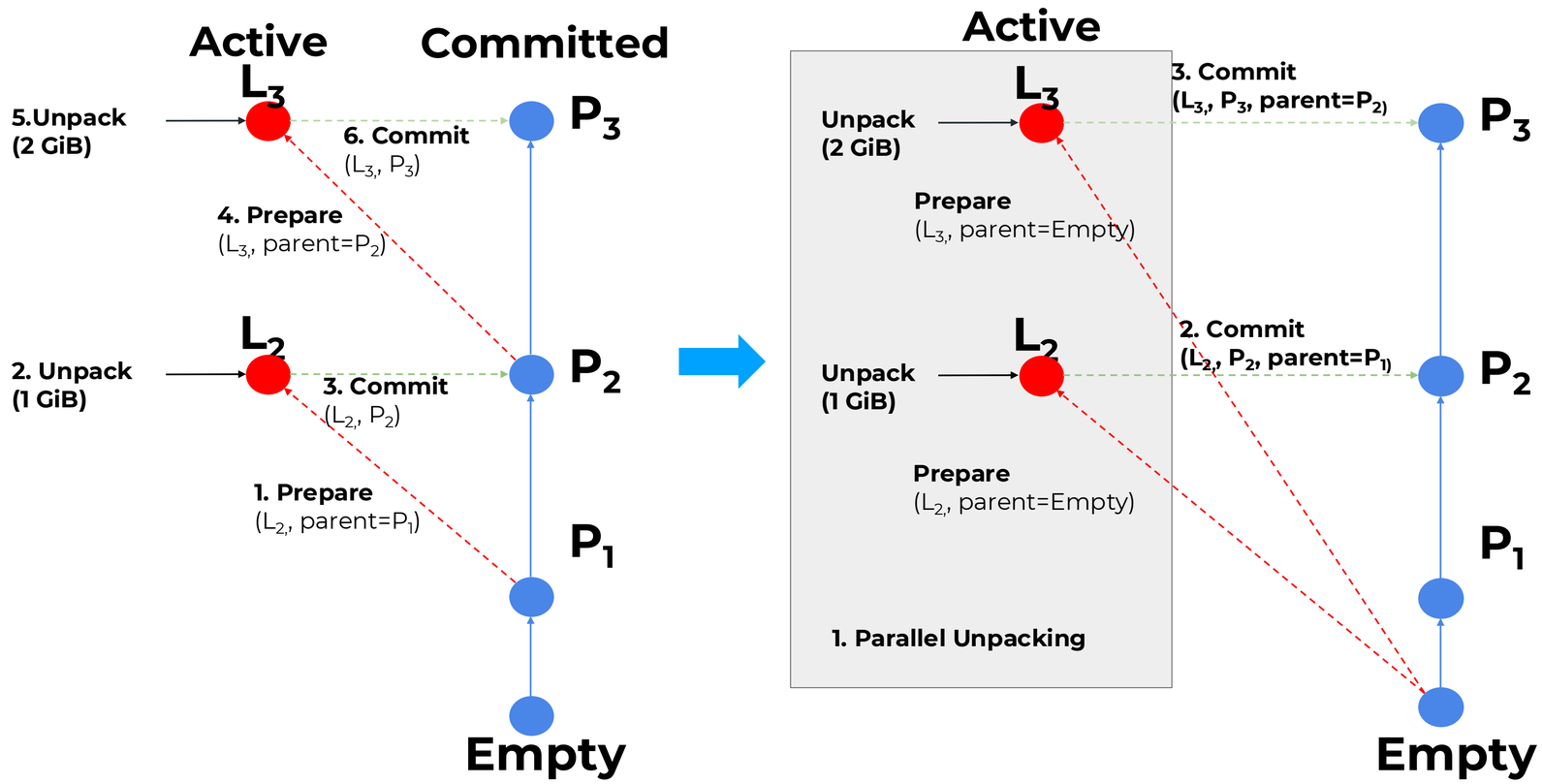
- Parallel Unpacking
- Added support for writable layer quota;
- Added support for block files as writable layer for VM-based containers;
- Never mount on the hosts if VM-based secure containers are used

Some community feedback:

“… I see great benefits on some user's workloads with this in place (2-3 times speedup in launch time) in my initial testing, so I'm eager to use this more widely…”

https://github.com/containerd/containerd/pull/12433

Parallel Unpacking



EROFS fsmerge feature

```
# Disk DescriptorFile
version=1
CID=fffffffe
parentCID=ffffffff
createType="twoGbMaxExtentFlat"

# Extent description
RW 4312 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/112/fsmeta.erofs" 0
RW 176520 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/92/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/93/layer.erofs" 0
RW 549528 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/94/layer.erofs" 0
RW 8 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/95/layer.erofs" 0
RW 97736 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/96/layer.erofs" 0
RW 8 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/97/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/98/layer.erofs" 0
RW 22960 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/99/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/100/layer.erofs" 0
RW 87736 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/101/layer.erofs" 0
RW 40 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/102/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/103/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/104/layer.erofs" 0
RW 106720 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/105/layer.erofs" 0
RW 88224 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/106/layer.erofs" 0
RW 24 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/107/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/108/layer.erofs" 0
RW 336 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/109/layer.erofs" 0
RW 135208 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/110/layer.erofs" 0
RW 24 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/111/layer.erofs" 0
RW 16 FLAT "/var/lib/containerd/io.containerd.snapshotter.v1.erofs/snapshots/112/layer.erofs" 0


# The Disk Data Base
#DDB

ddb.virtualHWVersion = "4"
ddb.geometry.cylinders = "1260"
ddb.geometry.heads = "16"
ddb.geometry.sectors = "63"
ddb.adapterType = "ide"
```

Considering container root filesystems for VM-based containers:

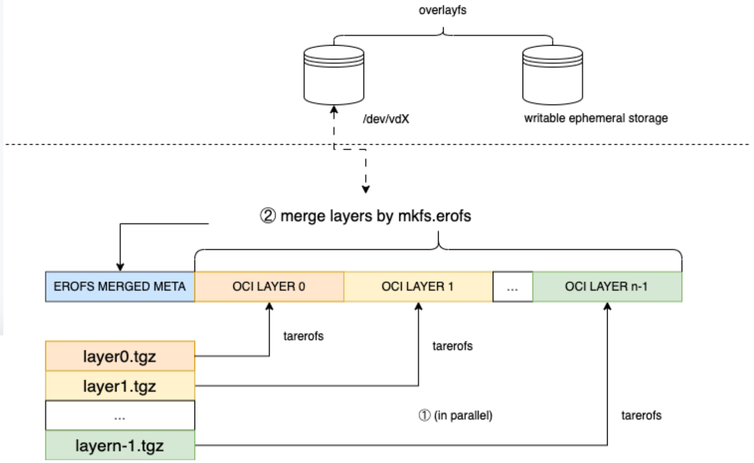
- Using virtio-fs opens FDs on the host, which can cause issues in cloud environments.
- Virtio-fs also introduces metadata performance challenges.

A better approach is to pass the container rootfs through directly as block devices.
Previously, each layer required its own EROFS mount or block device, which was inconvenient for VM-based containers.
EROFS can now merge all layers into a single mount with a single block device, but most VMMs still do not support multi-part block devices.

```
$sudo ctr run --runtime "io.containerd.nerdbox.v1" --snapshotter=erofs --rm --tty docker.io/library/wordpress:6.0 erofs lsblk
WARN[0000] DEPRECATION: The support for cgroup v1 is deprecated since containerd v2.2 and will be removed by no later than May
.
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
vda  254:0    0     5G  0 disk
vdb  254:16   0 620.2M  1 disk
```

writable layer with 5GiB disk quota

merged EROFS container image blkdev

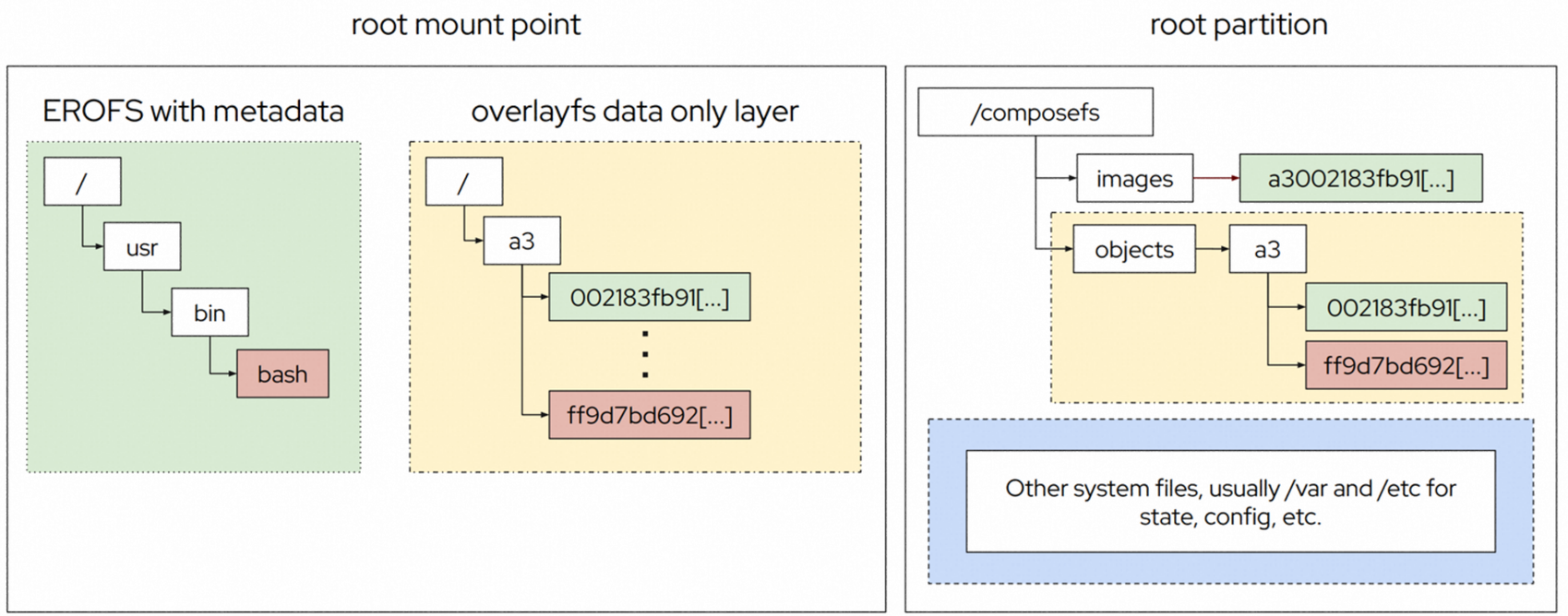


EROFS fsmerge feature



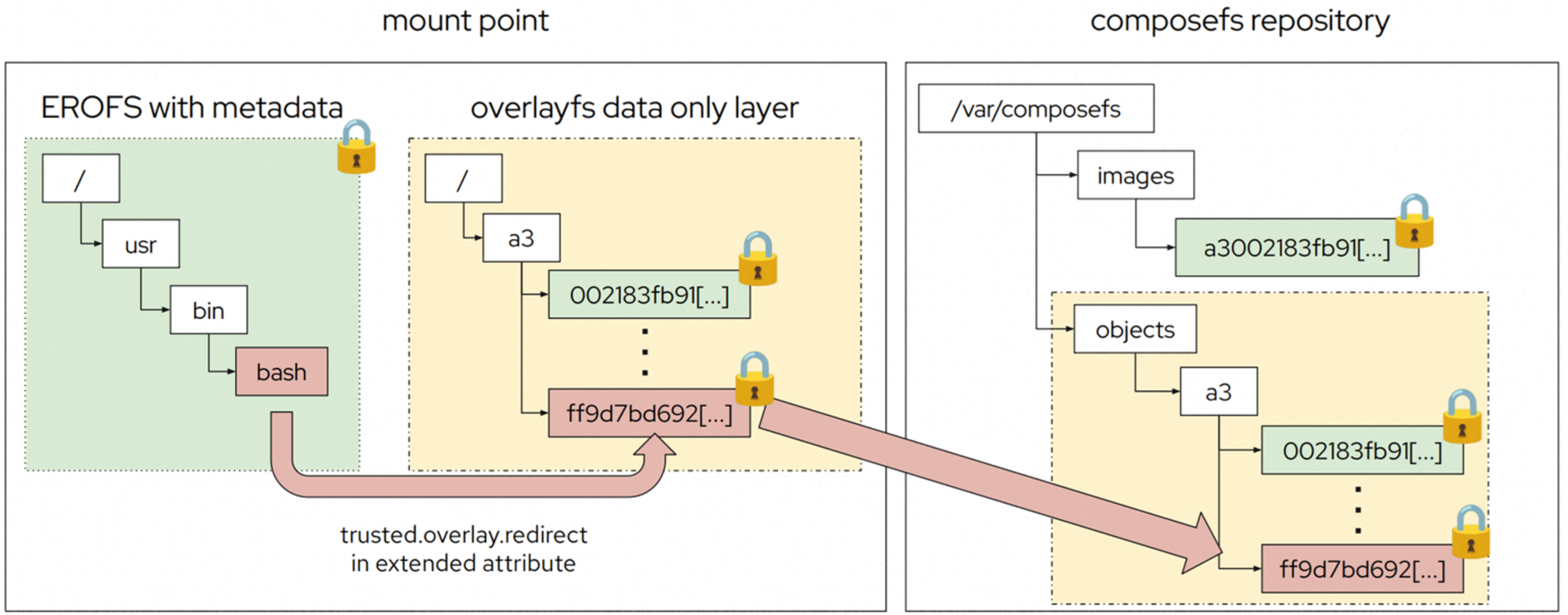
ComposeFS

Use EROFS + OverlayFS to improve OSTree to overcome hard-link issues (e.g. unexpected shared metadata, no data verification)



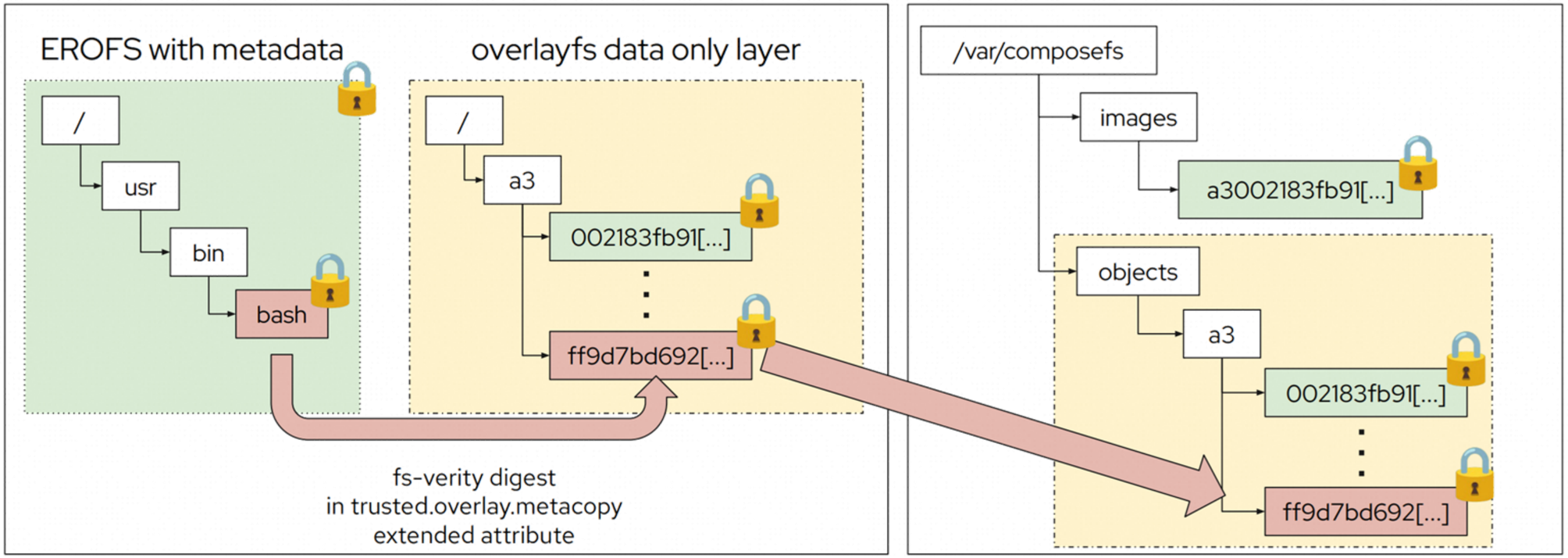
ComposeFS

Use overlayfs “trusted.overlay.redirect” xattr to redirect a file in EROFS metadata to a OSTree conten-addressable storage blob



ComposeFS

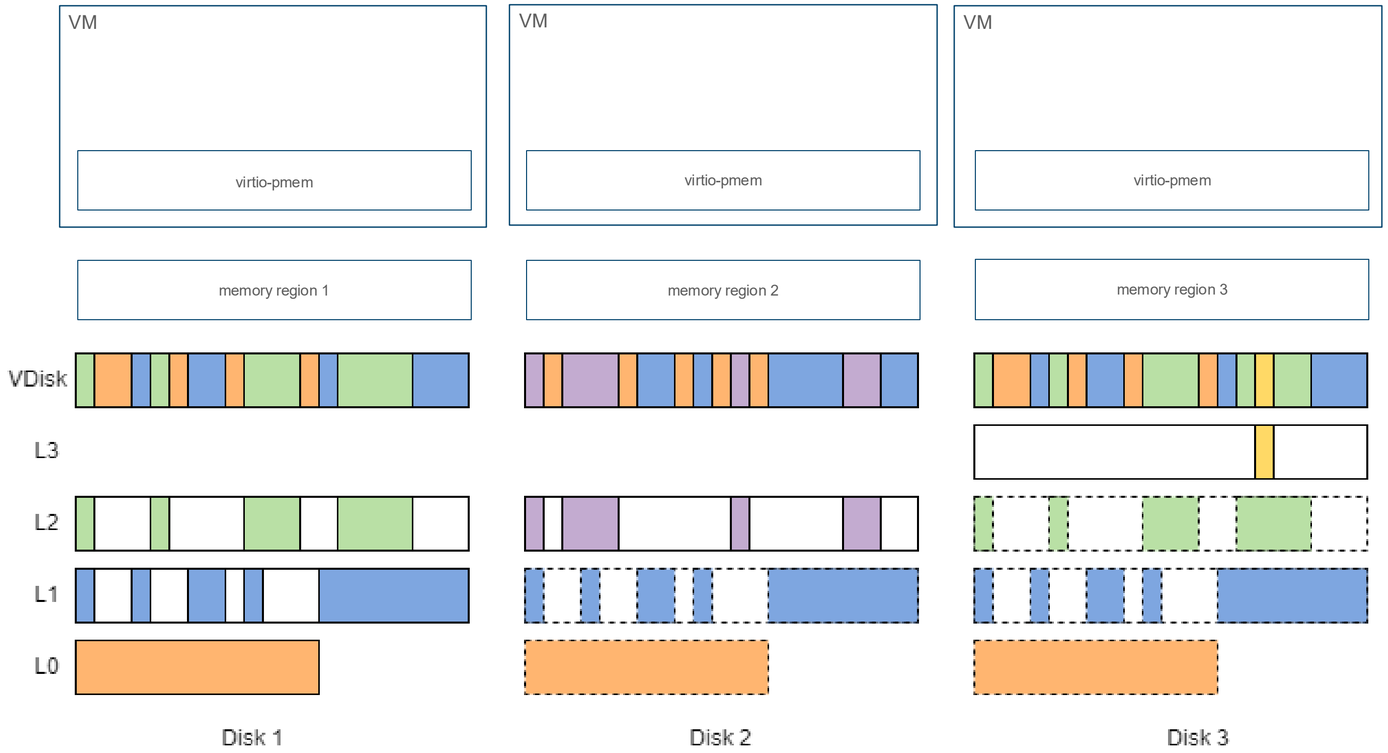
Overlayfs fs-verity digest is kept in “trusted.overlay.metacopy” to ensure the data integrity.



Finer-granularity (layer-based) FSDAX memory access

Problem

Disk-snapshot-like FSDAX with traditional FSes can only do image-level de-duplication conveniently.



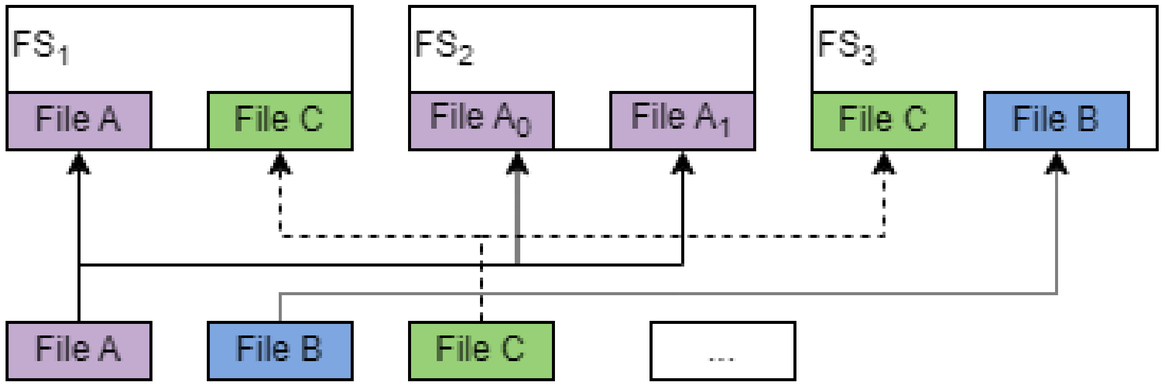
- Disk-snapshot-like FSDAX memory access will have N-times extra memory overhead due to minor image update;
- It will become a burden to maintain another 4k-mapping table for snapshot-like FSDAX memory access in order to try to fix that.

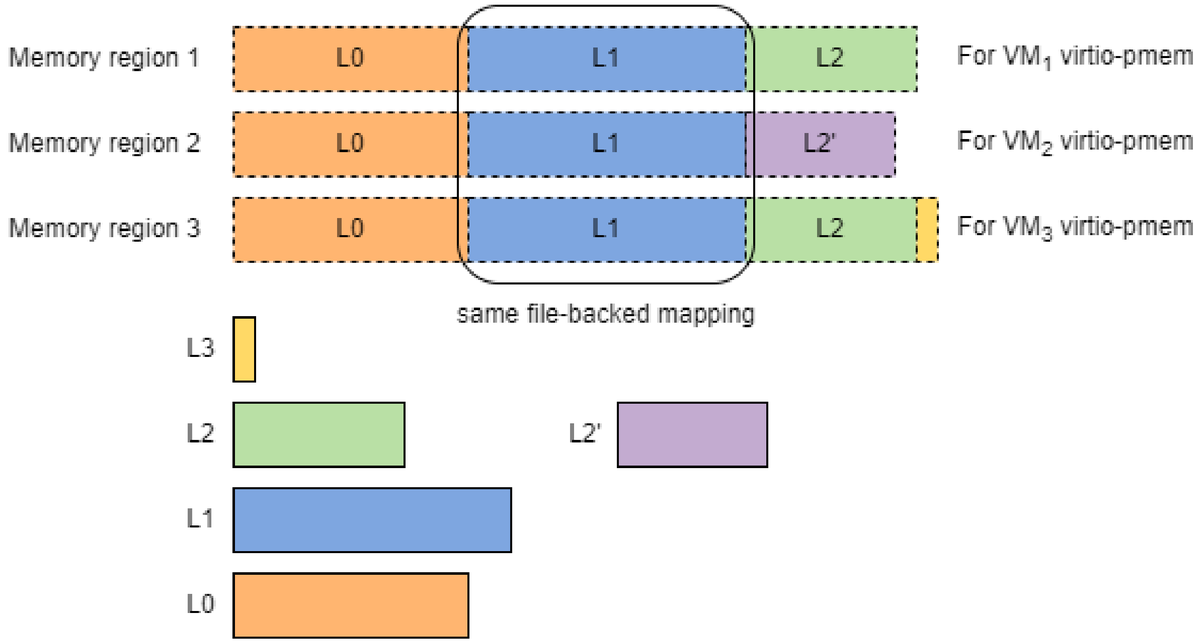
Finer-granularity (layer-based) FSDAX memory access

Solution

EROFS supports sub-image merging;

Each PMEM memory region can be formed by multiple sub-images.





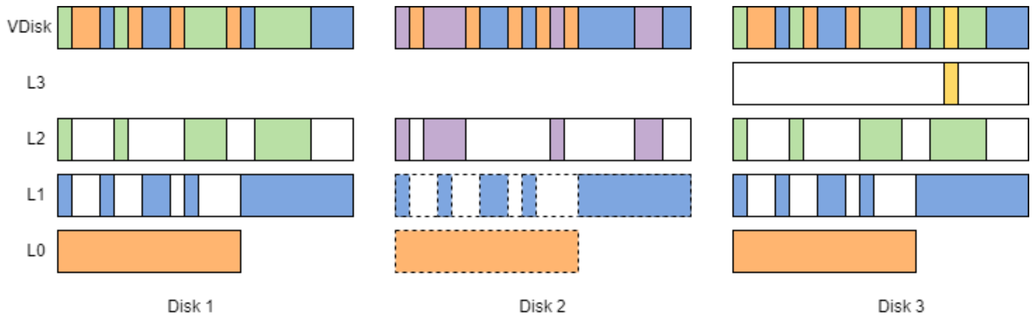
runc page cache sharing

Problem

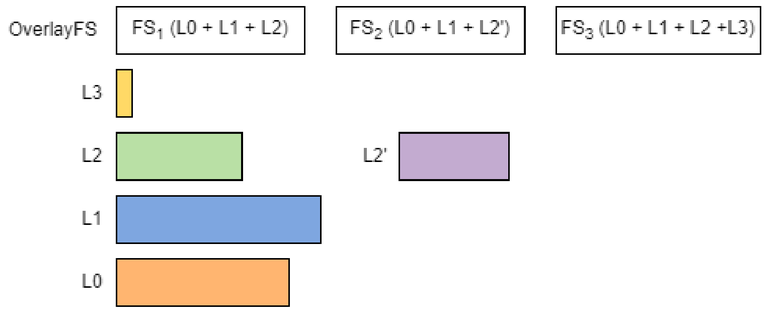
Snapshot-like approaches with traditional FSes can hardly share page cache on RunC:

Even compared to OverlayFS (layer-level page cache sharing);

OverlayFS itself cannot share page cache on the same file across different layers:



Disk snapshots



Overlayfs layer sharing

runc page cache sharing

Solution

- Each on-disk inode can be assigned with a unique fingerprint (secure hash or fsverity digest for example);
- Inodes with the same fingerprint will share page cache data using anonymous inodes on the node;
- Some backing file infrastructure is used for users to make file_{user_path,user_inode};
- From my own perspective, no major issue so far, aims to land this in the next cycle

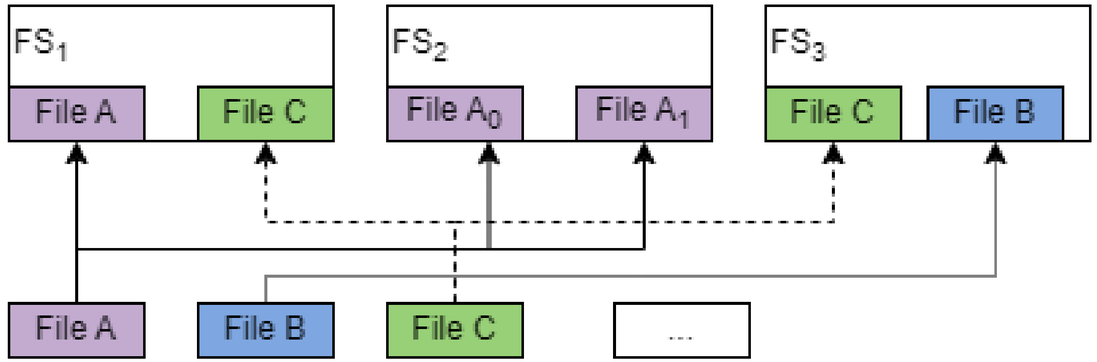


Image	Page Cache Share	Memory (MB)	Memory Reduction (%)
redis 7.2.4 & 7.2.5	No	241	-
	Yes	163	33%
postgres 16.1 & 16.2	No	872	-
	Yes	630	28%
tensorflow 2.11.0 & 2.11.1	No	2771	-
	Yes	2340	16%
mysql 8.0.11 & 8.0.12	No	926	-
	Yes	735	21%
nginx 7.2.4 & 7.2.5	No	390	-
	Yes	219	44%
tomcat 10.1.25 & 10.1.26	No	924	-
	Yes	474	49%

points for discussion



FS_USERNS_MOUNT

Not same as unprivileged automount because of userns mounts are isolated by new user namespaces, but the security requirement of the on-disk format is the same.

Very strong requirement for containers, but popular Linux disk filesystems like EXT4, XFS resist this feature:

What I can see from the generic perspective of filesystem engineering:

1） On-disk format with redundant metadata makes them hard to verify if those additional filesystem metadata is consistent or not without noticeable performance loss;

e.g. the allocation status of a physical block may be recorded in multiple places: the allocation tree (or bitmap), inode extents, and may also appear in a reverse mapping tree. When an image is fetched from an untrusted remote source, an attacker can craft inconsistencies and the resulting bugs are serious and hard to prevent.

filesystem journaling makes it even more complex to achieve.

2) runtime implementation for generic filesystems also complicates this issue (e.g. no strict realtime consistency check for performance) : no development resource to address security issues actively and in time;

FS_USERNS_MOUNT

But for simple filesystems like immutable filesystems, their on-disk format parsing seems to be even simpler than FUSE protocol itself —— FUSE becomes more and more complicated over years and could become a potential source of vulnerabilities too.

while various other new “simpler fs” attempts on this from time to time, like the following statement:

“Yeah, EROFS is simpler than read-write OSes, but its not dead simple. The original in-kernel composefs was even simpler, and I believe something like that could eventually get fuzzed to be trustworthy for untrusted data.”

Which makes the game even more insane: because it leads to an unknown development direction: people reinvent new filesystems (reduced LOCs, use Rust, or other whatever tricky ways) to just try to achieve this only goal, which wastes the overall development resource and causes even more churn in the community.

People also think since a filesystem doesn’t support USERNS_MOUNT, it cannot be claimed to resist malicious remote images also from the userspace point of view: it’s even hard to get an agreement among fragmented userspace container engines just because the kernel community cannot get an agreement.

Why it cannot become a fs-specific policy instead? Use a dedicated Kconfig just like CONFIG_USER_NS itself and limit to the core on-disk format. Or other alternative kernel approach?

FSDAX page mapping sharing issue when applying to EROFS

1) EROFS data(chunk-based) deduplication cannot be applied to FSDAX

— The current FSDAX only supports shared FSDAX page mapping with filesystem reverse mapping tree;

2) It’s unnecessary for EROFS to introduce a dedicated on-disk filesystem reverse mapping tree to archive block data sharing;

3) The on-disk reverse mapping tree is another redundant metadata, which can be compromised by malicious source;

Hard to check this kind of metadata consistency easily or costly.

See comments in the original dax reflink patchset (Apr 2020):

https://lore.kernel.org/all/20200427084750.136031-1-ruansy.fnst@cn.fujitsu.com/

DM-verity limitation

Although the block-based markle tree doesn't in priciple, but dm-verity itself is currently based on the device mapper infrastructure:

- So that EROFS file-based mounts cannot be benefited from dm-verity;

- The markle tree cannot be applied to all external blobs(devices): have to apply them one-by-one and broken.

- It cannot be used for FSDAX use cases, for example, memory maped I/Os should be verified during page fault.

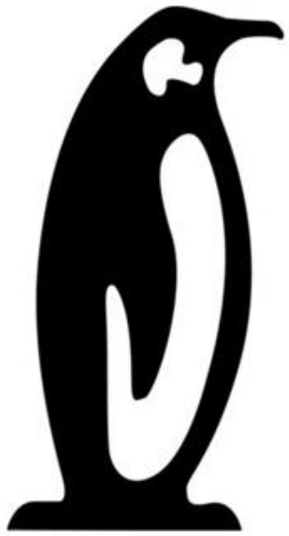
It doesn't have relationship with the current fsverity, since it's only used to protect individual files in a filesystem, and the markle tree is in the out-of-band area.

Solution

If it is possible to form out a common libverity merkle tree library to make filesystem to easy to use;

So that EROFS or other filesystems can call into libverity directly to verify filesystem I/Os;

Also resolve FSDAX verification issues



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

